Design Document

for

Real-Time Multi-Vendor Food Ordering and Delivery Management System for a University Campus

Version 1.0

Prepared by

Group #:24 **Group Name: AlgoMinds**

Aarsh Kaushik	220014	aarshk22@iitk.ac.in
Vikas Saini	231151	svikas23@iitk.ac.in
Kajal	230513	kajalb23@iitk.ac.in
Banoth Ramcharan	240259	ramcharanb24@iitk.ac.in
Ritik Ahirwar	210862	ritika21@iitk.ac.in
Banoth Venkatamaniram	240260	banothv24@iitk.ac.in
Abhimanyu Singh Rawat	240029	abhimanyur24@iitk.ac.in
Ashish Kuman Soni	230228	sashish23@iitk.ac.in
Pravin Tanpure	220815	pravint22@iitk.ac.in

Course: **CS253**

Mentor TA: ***Aditya Ojha***

Date: **06/02/2026**

CONTENTS II

REVISIONS II

1 CONTEXT DESIGN 1

1.1 CONTEXT MODEL 1

1.2 HUMAN INTERFACE DESIGN 1

2 ARCHITECTURE DESIGN 2

3 OBJECT-ORIENTED DESIGN 3

3.1 USE CASE DIAGRAM 3

3.2 CLASS DIAGRAM 3

3.3 SEQUENCE DIAGRAM 3

3.4 STATE DIAGRAM 3

4 PROJECT PLAN 4

5 OTHER DETAILS 5

APPENDIX A - GROUP LOG 6

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
0.1	Aarsh Kaushik, Vikas Saini	Initial draft of the Software Design Document created. Basic project overview, scope, and preliminary system description added.	25/01/2026
0.3	Kajal, Banoth Ramcharan	User interface concepts and high-level system architecture discussed and documented. Initial task distribution finalized	25/01/2026
0.5	Ritik Ahirwar, Banoth Venkatamaniram	Core system classes identified. Architectural design refined and high-level algorithmic flow documented.	30/01/2026
0.7	Abhimanyu Singh Rawat, Ashish Kumar Soni	Use case diagrams, sequence diagrams, and state diagrams completed and reviewed. Document structure validated.	03/02/2026
0.9	Pravin Tanpure	Final refinements to diagrams, consistency checks, formatting corrections, and content validation performed.	05/02/2026
1.0	All Group Members	Final version of the Software Design Document completed after incorporating all feedback and final reviews.	05/02/2026

1 Context Design

1.1 Context Model

The campus food ordering and delivery system operates as a centralized software platform within a university environment. It interacts with multiple external human actors, including students and faculty who place and track food orders, food vendors who manage menus and process orders, and delivery personnel who receive delivery assignments and update delivery status. In addition to human actors, the system interacts with external services such as a third-party payment gateway for processing online payments and a mapping service for location visualization and real-time delivery tracking. All internal processing, data management, and communication logic are encapsulated within the system boundary, which is treated as a black box in this context model.

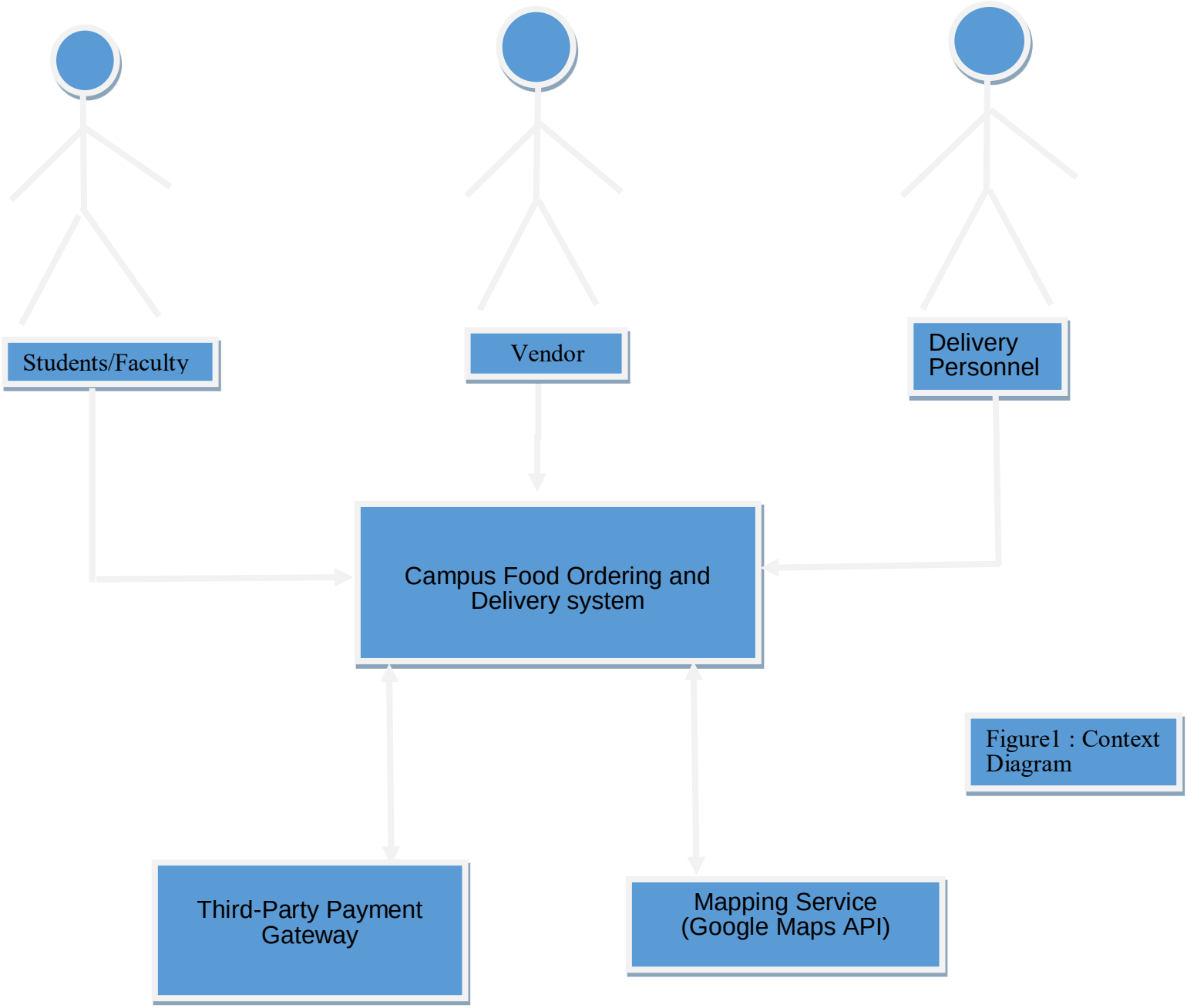


Figure1 : Context Diagram

Interaction Summary:

- Students/Faculty ↔ System: Place orders, view status, track deliveries, make payments
- Vendors ↔ System: Receive orders, update order status, manage menus
- Delivery Personnel ↔ System: Receive assignments, update delivery status
- System ↔ Payment Gateway: Payment requests and confirmations
- System ↔ Mapping Service: Location and route data exchange

1.2 Human Interface Design

1. User Interface – Students & Faculty

The user interface for students and faculty is designed to enable quick and intuitive food ordering with minimal interaction steps. Upon launching the application, users are presented with an authentication screen that allows them to log in or register. After successful authentication, users are directed to a vendor list screen displaying all available campus food outlets in a clear, scrollable list.
Selecting a vendor opens the menu screen, where food items are displayed along with their names, prices, and simple quantity controls. Users can add or remove items directly from the menu, and a cart summary is continuously accessible, allowing users to review selected items before confirming the order.
After an order is placed, users are redirected to a dedicated order tracking screen. This screen displays the current order status, such as accepted, preparing, or out for delivery, and provides real-time updates as the order progresses. Once the order is dispatched, the interface presents a map view that enables users to track the delivery personnel's location in real time. During this period, users are restricted to tracking the active order until it is completed or cancelled.
Payment, when required, is handled through a separate payment screen that redirects users to an external payment gateway and returns them to the application upon completion of the transaction. The interface prioritizes clarity, minimal navigation depth, and immediate feedback for user actions to ensure a smooth and reliable ordering experience.

2. Vendor Interface – Food Outlets

The vendor interface is designed as a dashboard-style web interface optimized for efficiency and rapid order processing. After authentication, vendors are presented with a home dashboard that summarizes incoming orders and provides access to menu management features.

The order management screen displays a list of current orders with clear status indicators. Vendors can select an order to view its details and update its status using simple controls such as accept, prepare, ready, or cancel. Status updates are reflected immediately in the system to notify users and delivery personnel.

The menu management screen allows vendors to add new menu items, update prices, and modify item availability through form-based controls. The interface is designed to minimize interaction complexity, allowing vendors to manage orders and menus with minimal effort during peak hours.

The vendor interface prioritizes visibility, speed, and ease of use to support continuous order handling.

3. Delivery Personnel Interface

The delivery personnel interface is designed for mobile use with a focus on simplicity and minimal user interaction. After authentication, delivery personnel are presented with a list of assigned delivery tasks, each showing essential information such as pickup location and delivery destination.

Selecting an assigned order opens a navigation screen displaying a map view for route guidance. Delivery personnel are not required to manually enter location data; location updates are handled automatically by the system. The interface provides simple controls to accept or reject delivery requests and to mark an order as delivered once completed.

The delivery interface is intentionally lightweight, reducing cognitive load and enabling delivery personnel to focus on navigation and timely order fulfillment.

2 Architecture Design

2.1 Proposed Architecture Overview

The proposed system follows a layered, service-oriented architecture with centralized backend services and multiple role-specific client interfaces. The architecture is designed to support real-time communication, scalability, and separation of concerns for a campus-wide food ordering and delivery system.

At a high level, the system consists of:

- **Client Layer** (User App, Vendor Dashboard, Delivery Module)
- **Backend Service Layer**
- **Data Layer**
- **External Services Layer**

All clients communicate with backend services using secure HTTP/HTTPS APIs and real-time WebSocket connections for live updates.

2.2 Architectural Components

1. Client Layer

This layer contains three client-facing applications:

- **User Application**: Used by students and staff to browse vendors, place orders, and track deliveries.
- **Vendor Dashboard**: Used by food vendors to manage menus and process incoming orders.
- **Delivery Module**: Used by delivery personnel to accept deliveries and share real-time location.

These clients handle presentation logic and minimal validation, while all core business logic resides in the backend.

2. Backend Service Layer

The backend is divided into modular services:

- **Authentication Service**
  Handles login, role-based access control, and session management for users, vendors, and delivery personnel.

- **Order Management Service**
  Responsible for order creation, status transitions, vendor updates, and delivery assignment.

- **Vendor Management Service**
  Manages vendor profiles, menus, item availability, and pricing.

- **Real-Time Communication Service**
  Uses WebSockets to propagate live order status updates and delivery location changes.

- **Notification Service**
  Sends real-time notifications to users, vendors, and delivery personnel on important events.

Each service is logically independent, enabling easier maintenance and future extension.

3. Data Layer

A **centralized database** stores:

- User profiles
- Vendor details and menus
- Orders and order states
- Delivery and location data

This layer ensures data consistency and integrity during concurrent access by multiple system actors.

4. External Services Layer

The system integrates with external services:

- **Mapping APIs** for GPS-based delivery tracking
- **Notification services** for push alerts and messages

These services are accessed through well-defined interfaces, keeping the core system decoupled from third-party dependencies.

2.3 Architectural Diagram

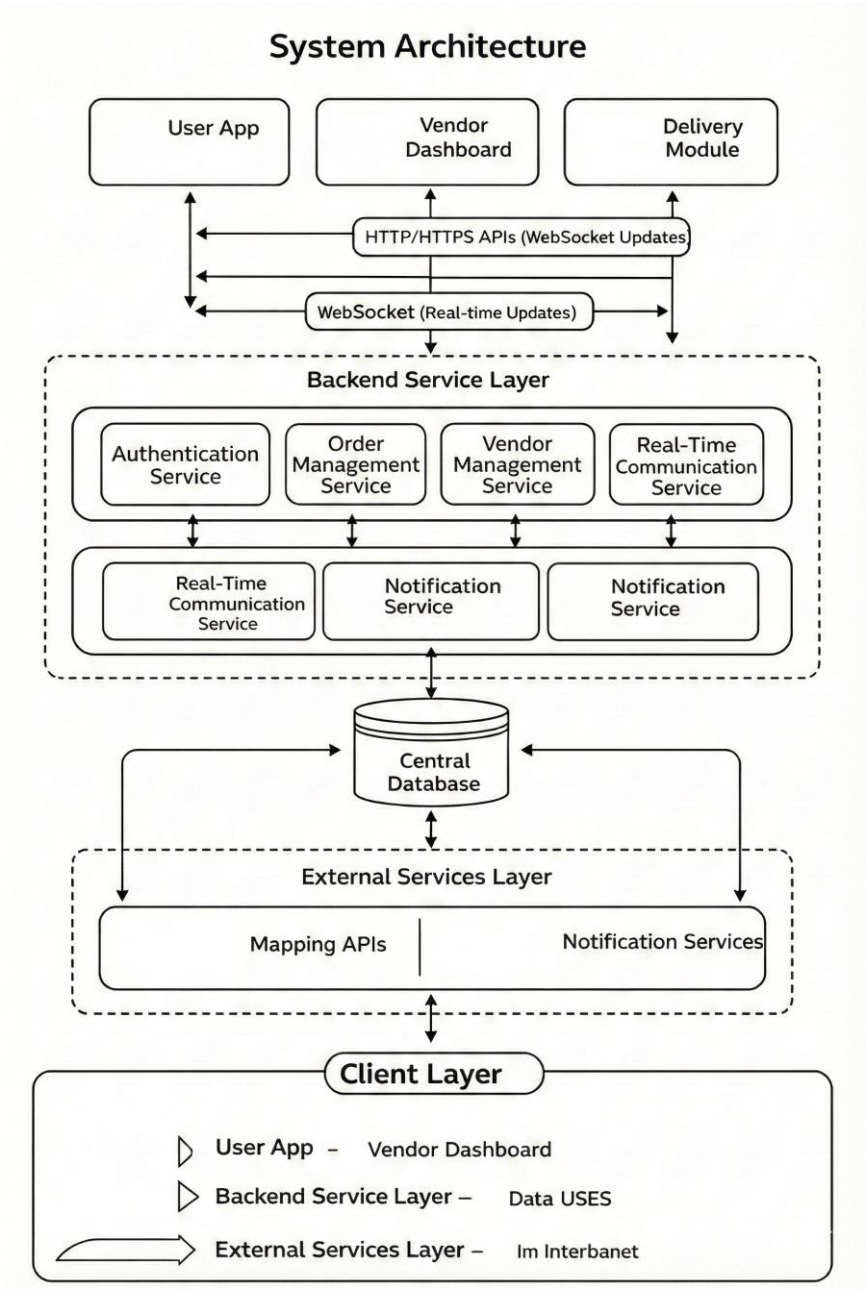


Figure 2 : Architecture Design

2.4 Intuition and Justification

Performance

- Real-time updates via WebSockets ensure order and location updates within required latency bounds.
- Centralized order processing minimizes redundant computations.

Scalability

- Modular backend services allow horizontal scaling as user load increases.
- Stateless service design supports concurrent users efficiently.

Reliability

- Centralized data storage prevents inconsistent order states.
- Service separation isolates failures and enables graceful degradation.

Maintainability

- Clear separation between presentation, business logic, and data layers simplifies debugging and future enhancements.
- Individual services can be updated independently.

Security

- Authentication and role-based access control restrict system access.
- Secure communication using HTTPS protects sensitive user data.

2.5 Conclusion

This layered, service-oriented architecture is well-suited for a real-time, multi-actor campus food ordering system. It balances performance, scalability, maintainability, and security while remaining flexible enough for future extensions such as payment integration or analytics modules.

3 Object Oriented Design

3.1 Use Case Diagrams

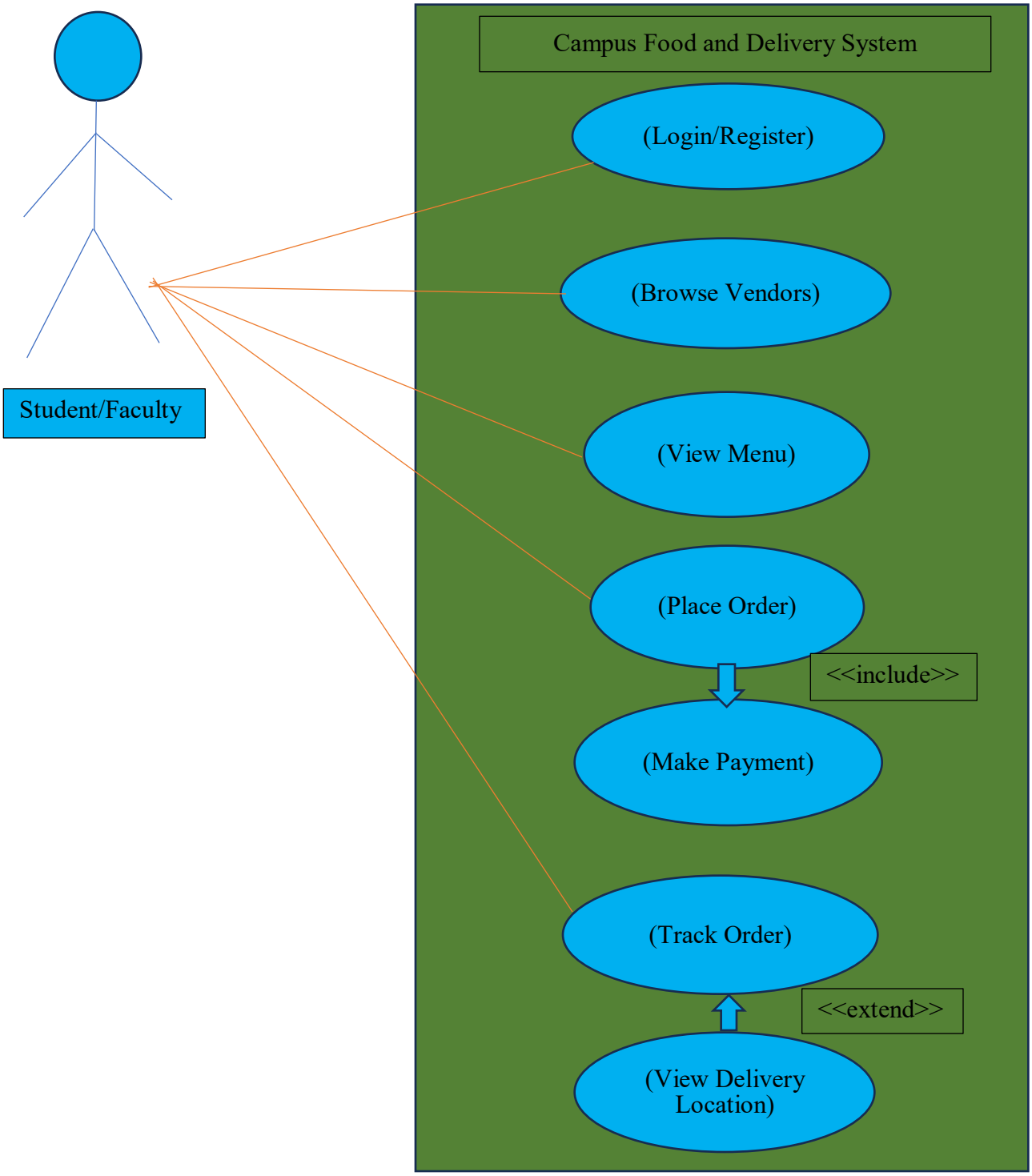


Figure 3 : Student / Faculty Use Case Diagram

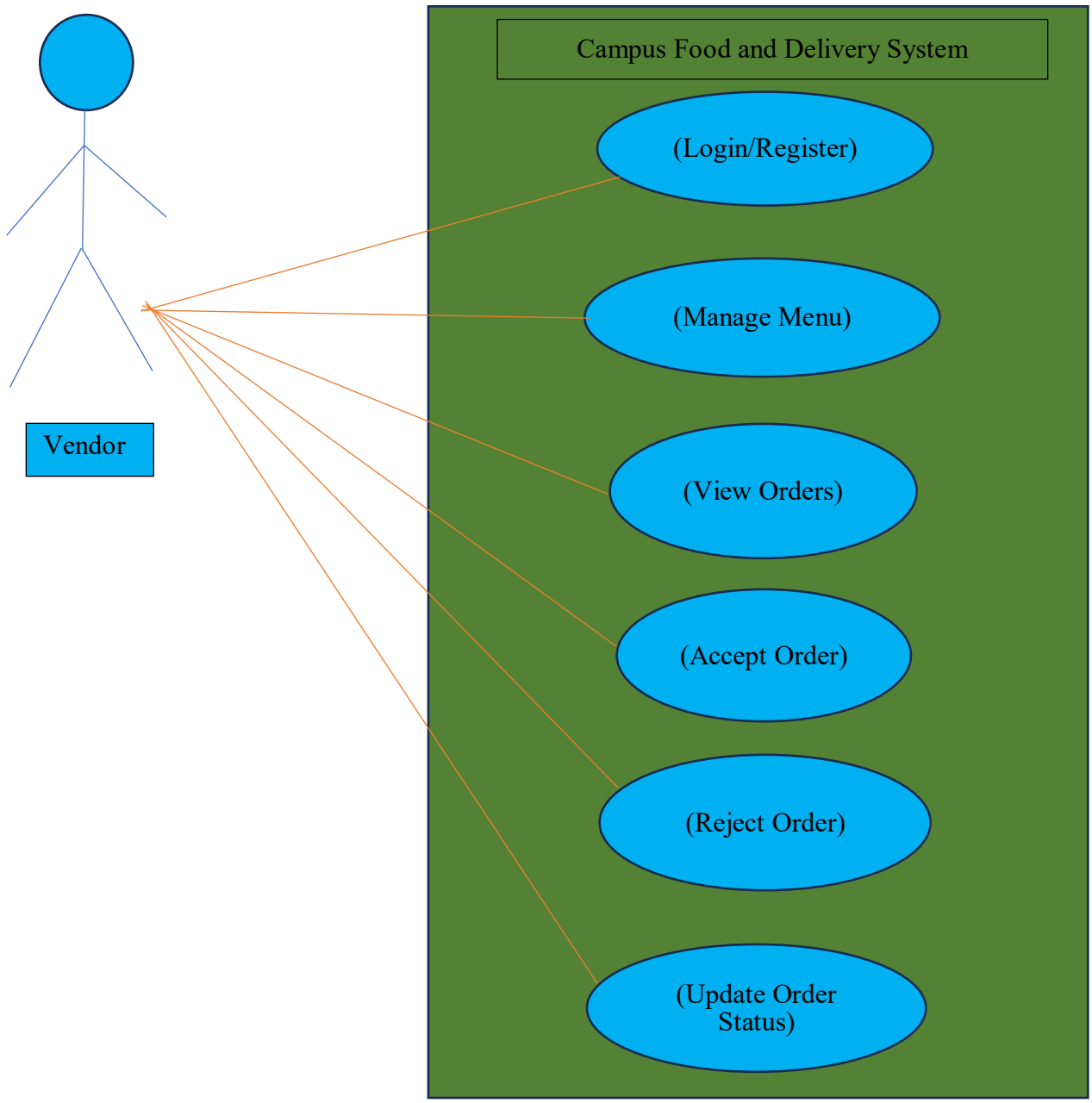


Figure 4 : Vendor Use Case Diagram

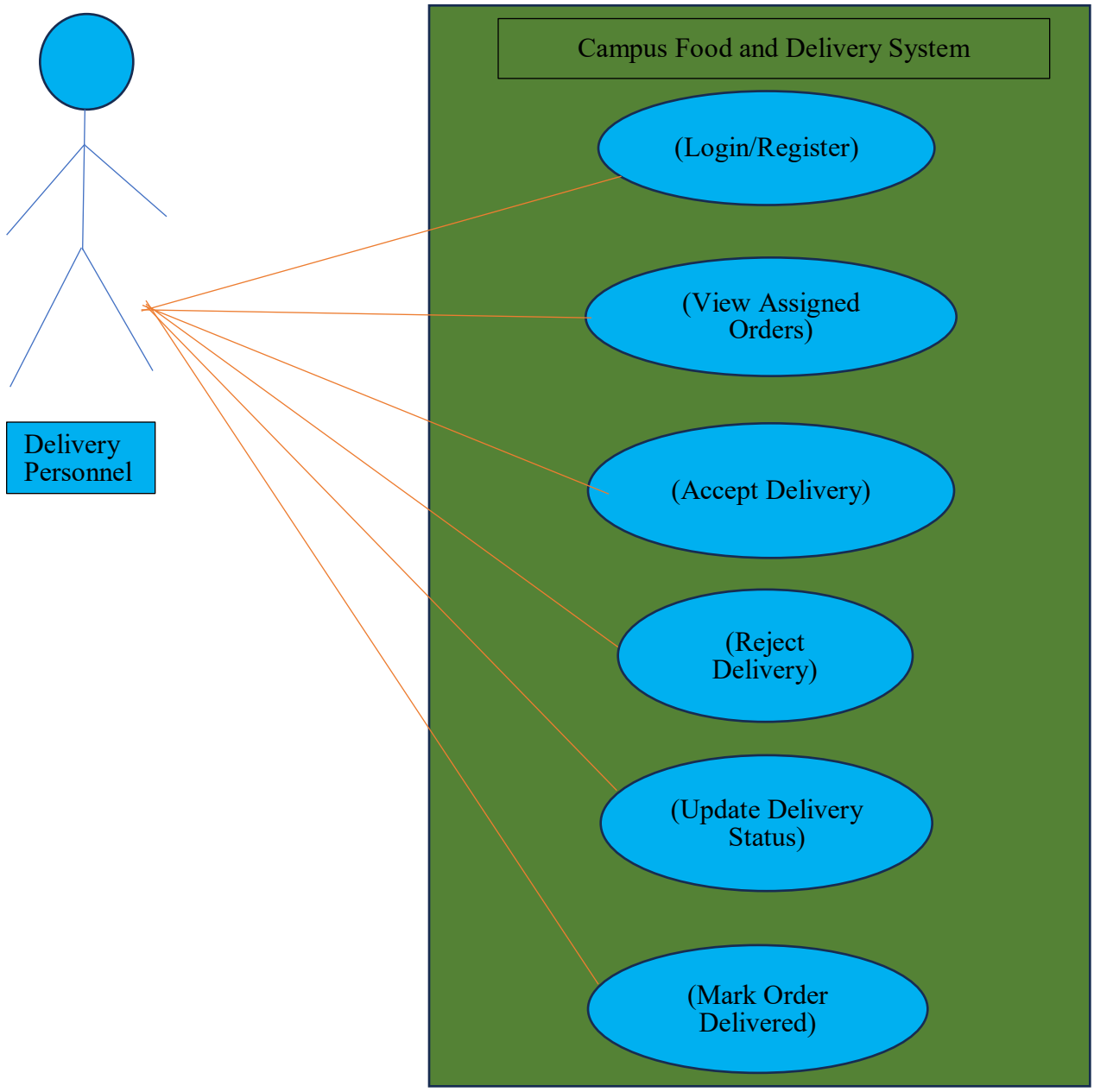


Figure 5 : Delivery Personnel Use Case Diagram

3.2 Class Diagrams

Class Attributes and Methods

User-related Classes

User
Represents a generic system user and contains common attributes required for authentication and identification.
Attributes: userId, name, email, phoneNumber, role
Methods: login(), logout()

StudentFaculty *(extends User)*
Represents campus users who place and track food orders.
Attributes: campusId, defaultLocation
Methods: placeOrder(), trackOrder(), makePayment()

Vendor *(extends User)*
Represents food outlets responsible for managing menus and processing orders.
Attributes: vendorId, vendorName, vendorLocation
Methods: updateMenu(), acceptOrder(), rejectOrder(), updateOrderStatus()

DeliveryPersonnel *(extends User)*
Represents delivery staff responsible for fulfilling delivery tasks.
Attributes: deliveryPersonId, availabilityStatus, currentLocation
Methods: acceptDelivery(), updateDeliveryStatus(), markDelivered()

Ordering and Menu Classes

Order
Represents a food order placed by a user.
Attributes: orderId, orderStatus, orderTimestamp, totalAmount
Methods: createOrder(), cancelOrder(), updateStatus()

OrderItem
Represents an individual food item within an order.
Attributes: quantity, itemPrice
Methods: calculateSubtotal()

Menu
Represents a vendor’s menu.
Attributes: menuId
Methods: getMenuItems(), updateMenu()

MenuItem
Represents a single food item offered by a vendor.
Attributes: itemId, itemName, price, availability
Methods: updateAvailability()

Payment and Delivery Classes

Payment
Represents payment details and status associated with an order.
Attributes: paymentId, paymentMethod, paymentStatus
Methods: processPayment(), verifyPayment()

Delivery
Represents the delivery process for an order.
Attributes: deliveryId, deliveryStatus
Methods: assignDelivery(), updateDeliveryStatus()

Location
Represents geographical information used for delivery tracking.
Attributes: address, latitude, longitude
Methods: updateLocation()

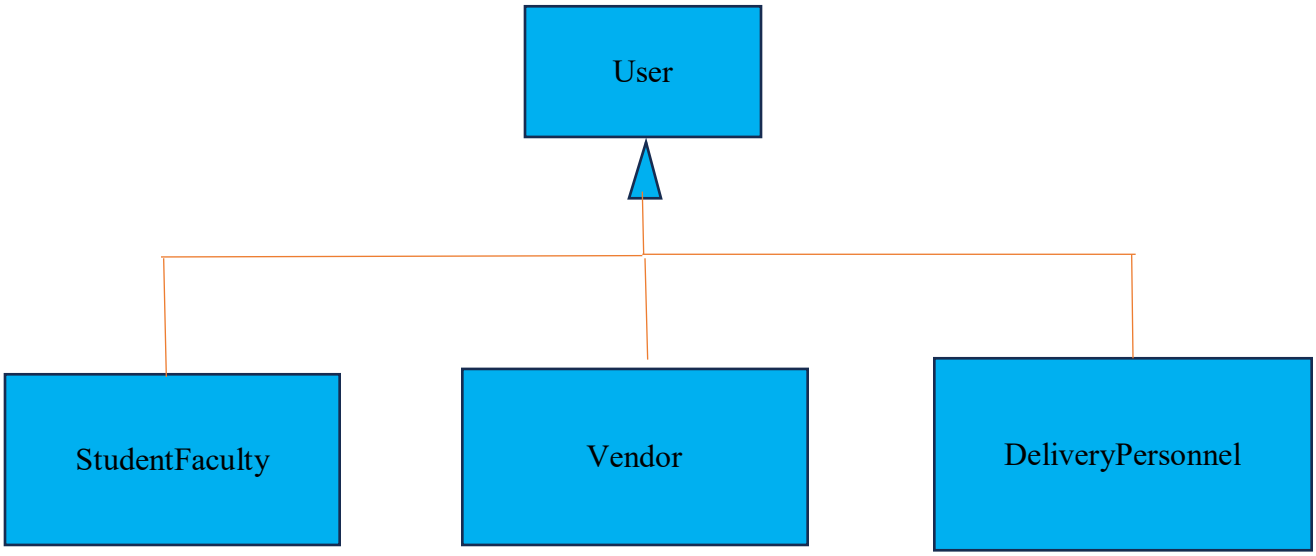
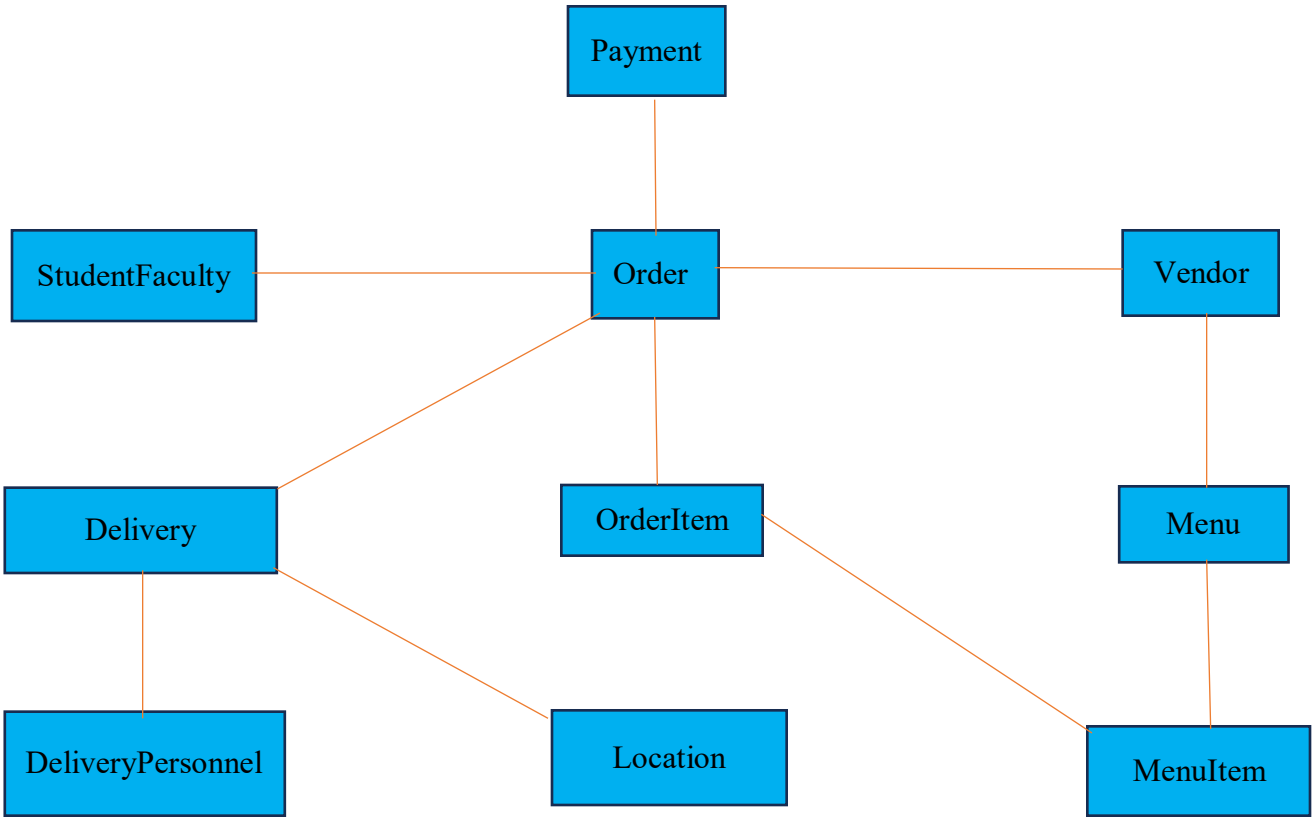


Figure 6 :
Inheritance Diagram



Relationship Description

- A **StudentFaculty** user can place multiple **Orders**, while each order is associated with exactly one student or faculty member.
- A **Vendor** can process multiple **Orders**, and each order is handled by a single vendor

.

- An **Order** consists of one or more **OrderItems**.
- Each **OrderItem** refers to a single **MenuItem**.
- A **Vendor** maintains a single **Menu**, which contains multiple **MenuItems**.
- Each **Order** is associated with one **Payment** and one **Delivery**.
- A **DeliveryPersonnel** can handle multiple **Deliveries**, while each delivery is assigned to a single delivery personnel.
- Each **Delivery** is associated with a single **Location**.

Figure 7 : Class Diagram

3.3 Sequence Diagrams

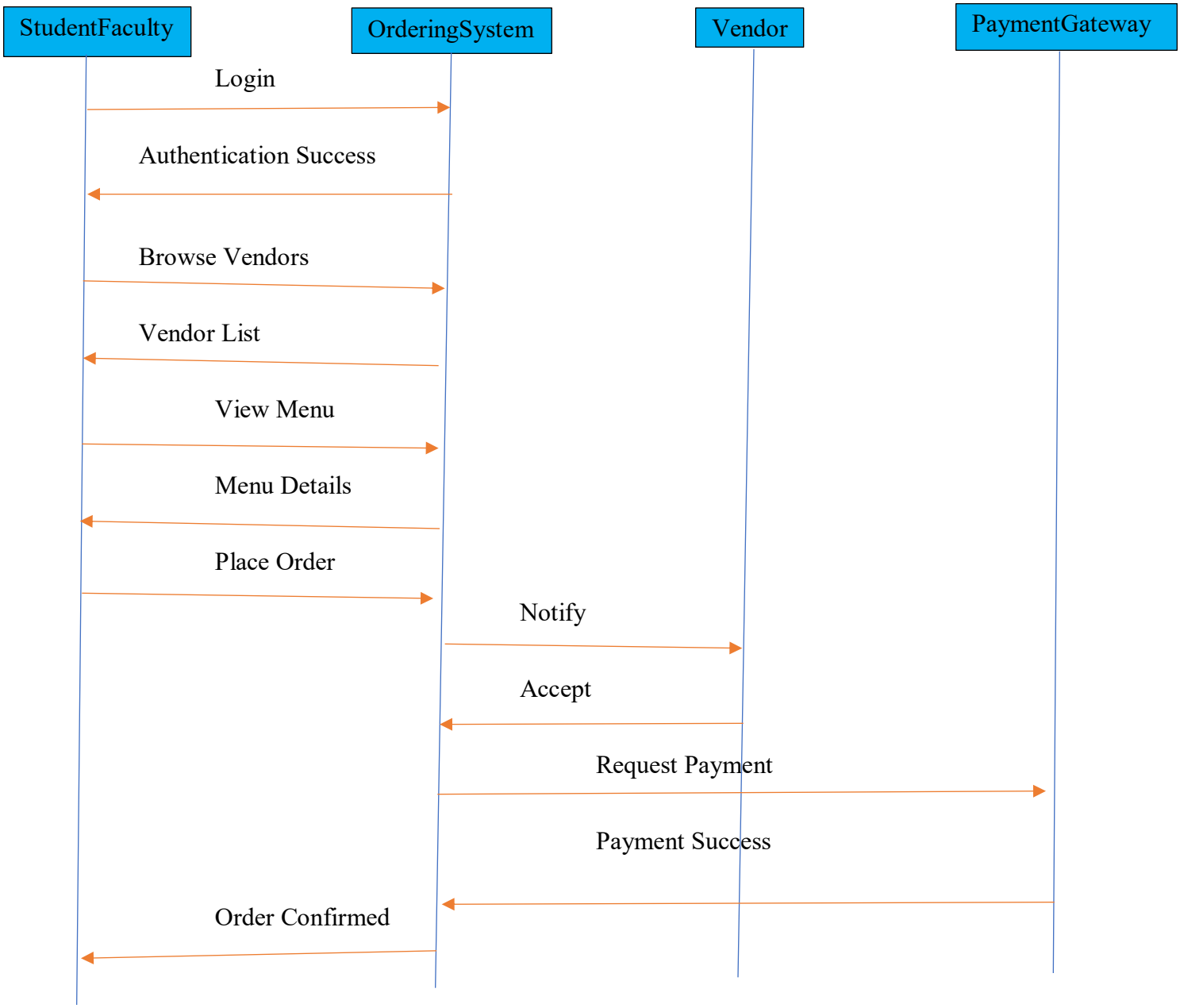


Figure 8 : Sequence Diagram : Place Order

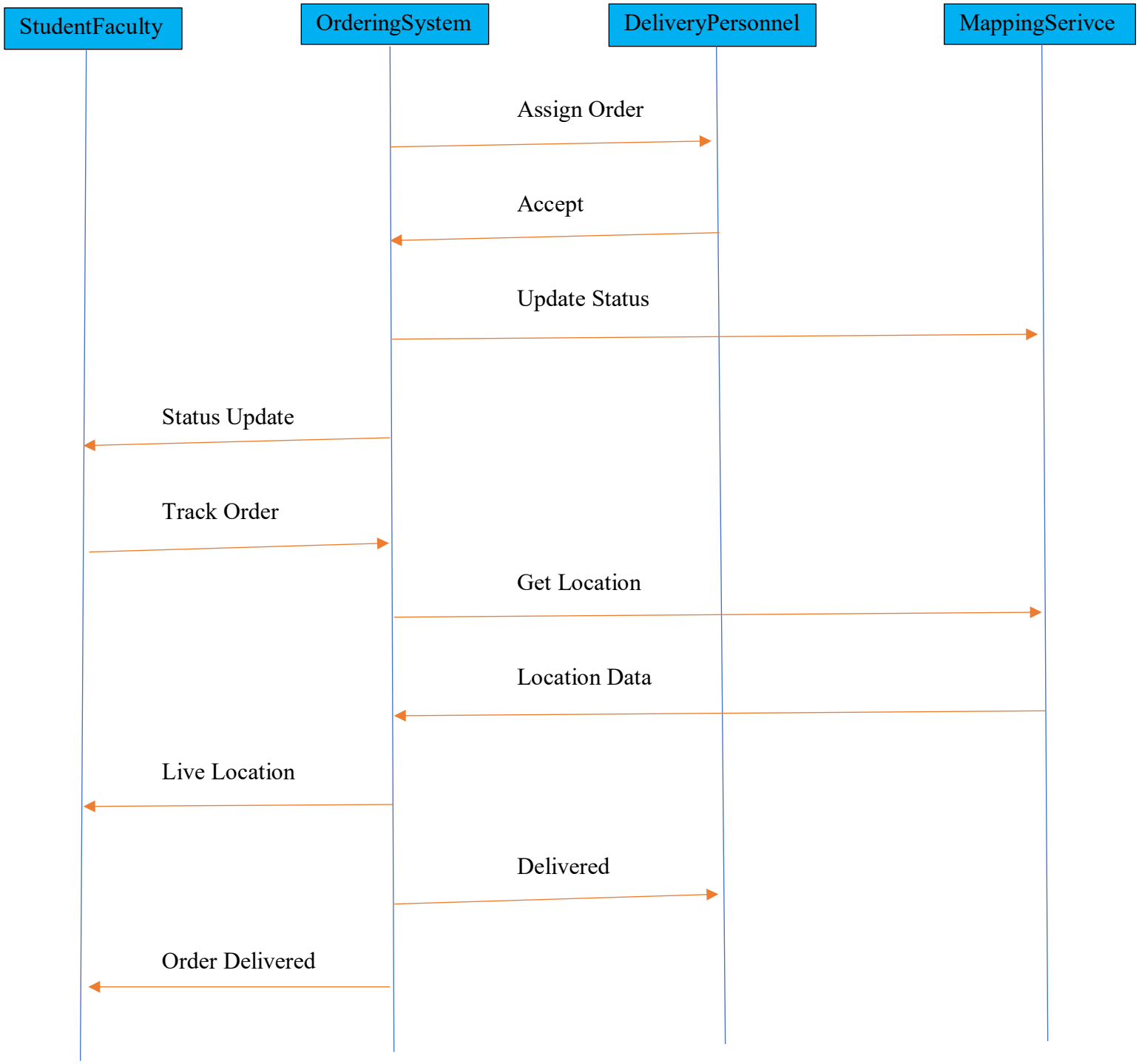


Figure 9 : Sequence Diagram : Delivery

3.4 State Diagrams

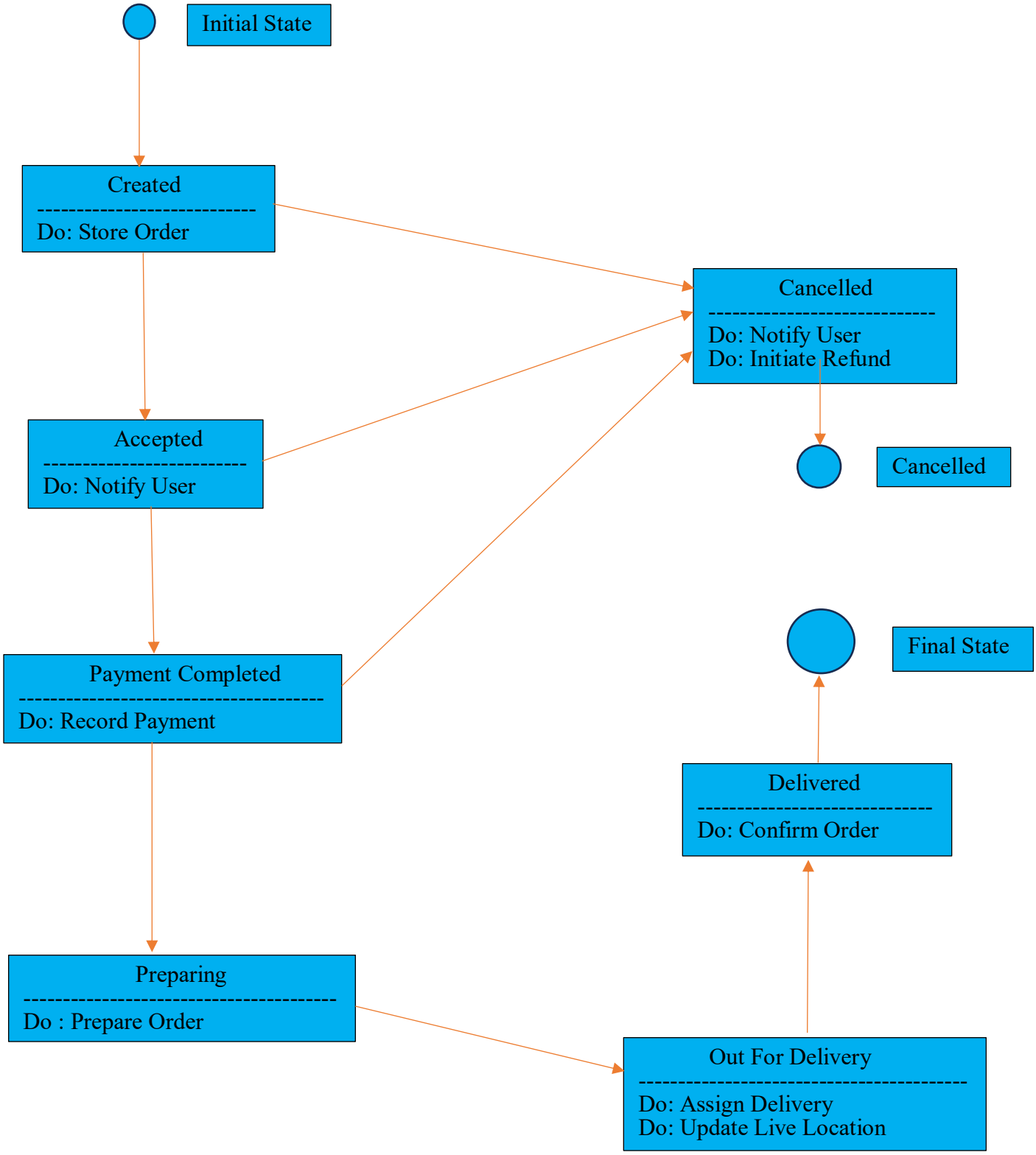


Figure 10 : State Diagram

The following transitions define the lifecycle of an order in the system:

1. **createOrder**
   Initial State → Created
   Triggered when a user successfully places an order.

2. **acceptOrder**
   Created → Accepted
   Triggered when the vendor accepts the order request.

3. **paymentSuccess**
   Accepted → Payment Completed
   Triggered when the payment is successfully processed.

4. **startPreparation**
   Payment Completed → Preparing
   Triggered when the vendor starts preparing the order.

5. **dispatchOrder**
   Preparing → Out for Delivery
   Triggered when the order is handed over to delivery personnel.

6. **markDelivered**
   Out for Delivery → Delivered
   Triggered when the delivery personnel confirms successful delivery.

Cancellation Transitions

The following transitions represent early termination of the order lifecycle:

7. **cancelOrder (before acceptance)**
   Created → Cancelled
   Triggered when the user cancels the order before vendor acceptance.

8. **cancelOrder (after acceptance)**
   Accepted → Cancelled
   Triggered when the order is cancelled after acceptance but before payment completion.

9. **cancelOrder (after payment)**
   Payment Completed → Cancelled
   Triggered when the order is cancelled after payment completion, with refund initiated as a side-effect.

Final Transitions

10. **completeOrder**
    Delivered → Final State
    Triggered when the order lifecycle is successfully completed.

11. **terminateCancelledOrder**
    Cancelled → Final State
    Triggered when cancellation processing is completed.

4 Project Plan

Click the link below to view the Project Plan

https://drive.google.com/file/d/1TLTdiDvxZLqRpamhaicWD9FBjI_o98Up/view?usp=sharing

Note: The project plan image is provided via hyperlink due to document rendering issues.

Project Phases:

UI–UX Design Phase

- **User Flow Mapping:**
  Defining the complete navigation flow for users, starting from registration/login to browsing food outlets, placing orders, tracking deliveries, and accessing order history.
- **Wireframing and Prototyping:**
  Creating low-fidelity wireframes for key screens such as login/registration, homepage, menu listing, cart, order tracking, and profile pages. High-fidelity prototypes were then developed to refine layout, usability, and user interaction.
- **Feedback Integration:**
  Reviewing UI designs based on usability considerations and campus user requirements to ensure simplicity, accessibility, and ease of use for students, vendors, and delivery personnel.

Backend Development Phase

- **Core Feature Architecture:**
  Designing and implementing backend logic for user authentication, role-based access (student, vendor, delivery agent), order processing, and status management.
- **Order and Delivery Management Engine:**
  Developing backend services to handle real-time order updates, vendor confirmations, delivery assignment, and order tracking.
- **Database Schema and API Development:**
  Designing the database schema to manage users, vendors, menus, orders, and

delivery data. RESTful APIs were implemented to enable seamless interaction between the frontend, vendor dashboard, and delivery module

Frontend Development Phase

- **Authentication Modules:**
  Implementing secure login and registration interfaces for different user roles with proper validation.
- **Dashboard and Homepage:**
  Developing intuitive dashboards for students, vendors, and delivery agents to manage orders, menus, delivery status, and profiles.
- **Feature Integration:**
  Integrating frontend components with backend APIs to enable real-time order placement, status updates, notifications, and delivery tracking.

Testing Phase:

- **Unit Testing:**
  Testing individual frontend and backend modules to ensure correct functionality.
- **Integration Testing:**
  Verifying smooth interaction between frontend, backend services, and database components.
- **System and Beta Testing:**
  Performing end-to-end testing of the complete system and validating performance, usability, and reliability under real usage scenarios.

Team Collaboration and Communication:

- **Agile Development and Peer Review:**
  The team followed an iterative development approach with tasks divided among members. Regular peer reviews ensured code quality and consistency across modules.
- **Communication and Version Control:**
  Regular meetings were conducted to track progress and resolve issues. Git was used for version control, with structured commits and pull requests to manage collaborative development effectively.

The following is the current work division that the team intends to follow:

MEMBERS	TASK
Aarsh Kaushik	Frontend Development, System Testing, Addressing Feedback
Vikas Saini	Full Stack Implementation, Code Improvement, Alpha Testing

Ritik Ahirwar	Backend Development, Integration Testing, UI/UX Design
Parvin Tanpure	Frontend Development, UI/UX Design, Testing
Abhimanyu Singh Rawat	Backend Development, System Testing, Addressing Feedback
Banoth Venkatamaniram	Full Stack Implementation, Code Improvement, Unit Testing
Kajal	Frontend Development, Unit Testing, UI/UX Design
Ashish Kumar Soni	Backend Development, Code Improvement, UI/UX Design

5 Other Details

Appendix A - Group Log

S. No.	DATE	TIMINGS	VENUE	DESCRIPTION
1.	25-01-2026	19:00-21:00	RM 3rd floor	Discussed the user interface design and overall system architecture for the campus food ordering and delivery system, and distributed the initial development work among team members.
2.	30-01-2026	20:00-22:00	RM 3rd floor	Reviewed the progress of the design document and finalized the system classes, architectural design, and a high-level overview of the algorithms to be used.
3.	03-02-2026	17:30-20:00	Google Meet	Completed the user interface design and finalized the state and sequence diagrams. The overall progress of the design document was reviewed and validated.
4.	05-02-2026	21:00-22:00	Google Meet	Finalized all remaining components and design aspects of the system documentation.